



第9章 面向对象实现

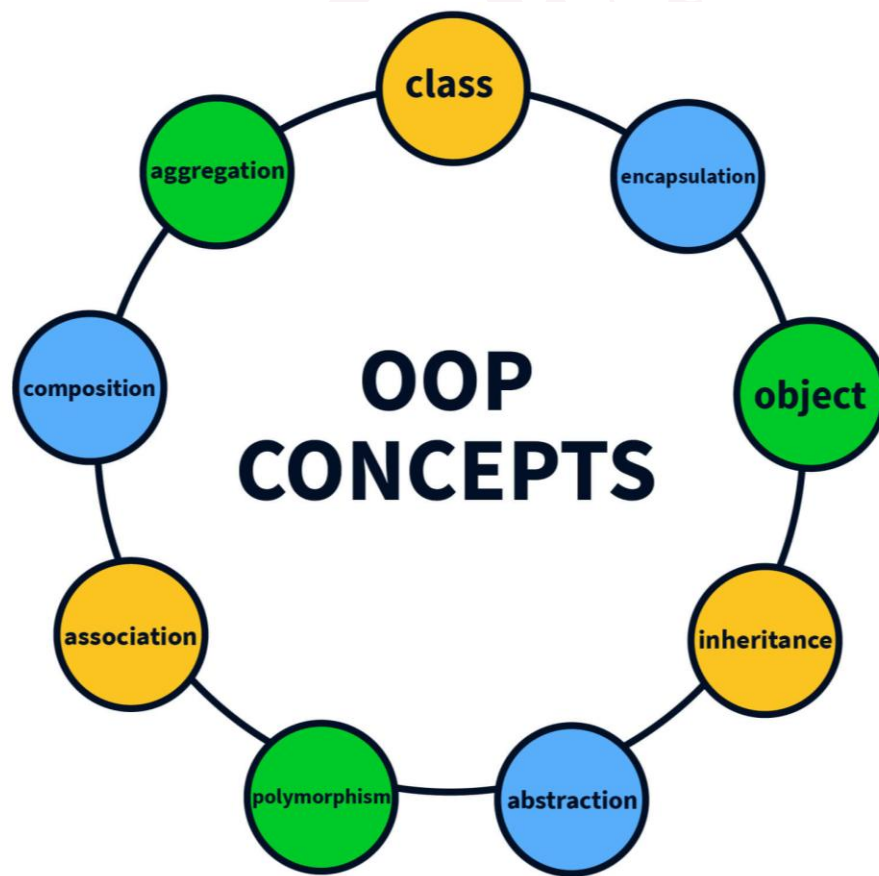
主要内容

面向对象编码

- 项目文件组织
- 面向对象业务实现

面向对象测试

- 面向对象测试阶段
- 面向对象测试策略
- 面向对象测试用例设计



1. 面向对象编码

[1] 项目文件组织

在面向对象设计中，软件设计人员已经对目标软件中涉及的类，以及类之间的交互进行了设计。同时，在进行构件设计时，软件设计人员可以根据需求将内容封装为构件，通过构件来实现目标软件。

如果在软件实现过程中将所有的内容细化为最小粒度，可以认为每个类的实现均为一个构件。

以C++程序为例： *.h *.cpp

1. 面向对象编码

项目文件组织实施：

步骤1：建立项目文件管理目录

步骤2：依次构建各个构件的包含内容

步骤3：编译并测试软件构件

步骤4：构件集成

步骤5：目标项目业务实现

1. 面向对象编码

步骤1: 建立项目文件管理目录

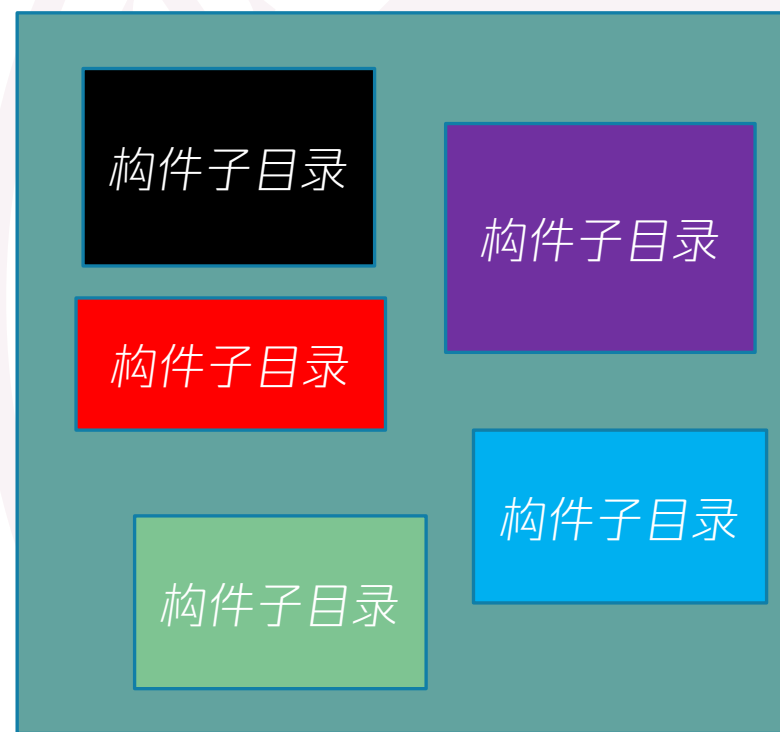
软件开发人员可以在本地目录中建立一个以项目英文名为名字的文件目录。

文件夹内部组织:

- ① 参考软件架构框架建议组织文件
- ② 结合软件设计来优化文件组织

1. 面向对象编码

软件开发人员可以根据面向对象设计，在项目文件目录中分别建立以构件名对应的子目录，并将该构件包含类的类界面和类实现存储于构件目录中。

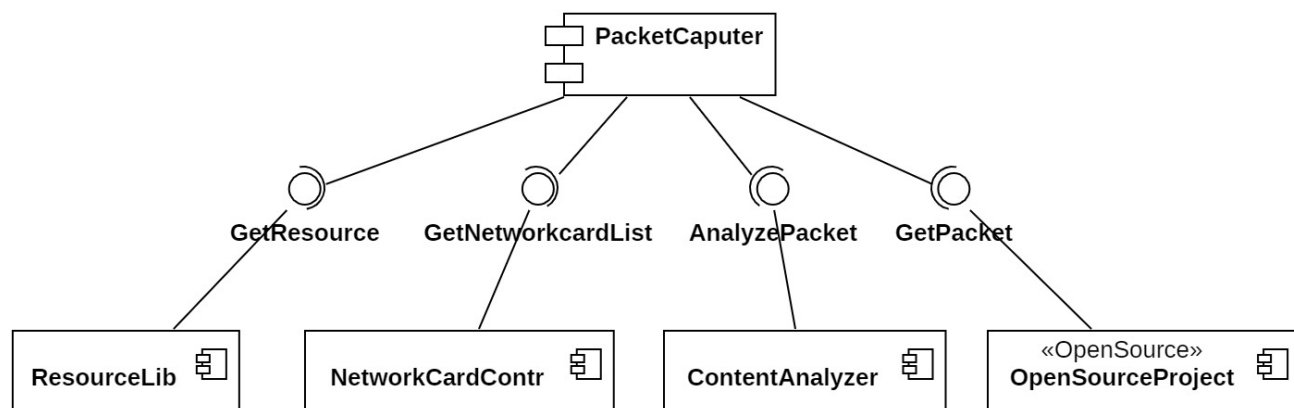


项目文件目录

1. 面向对象编码

抓取并分析网络数据包案例

软件目录结构实施案例



Data (D:) > PacketCaptuer >

名称

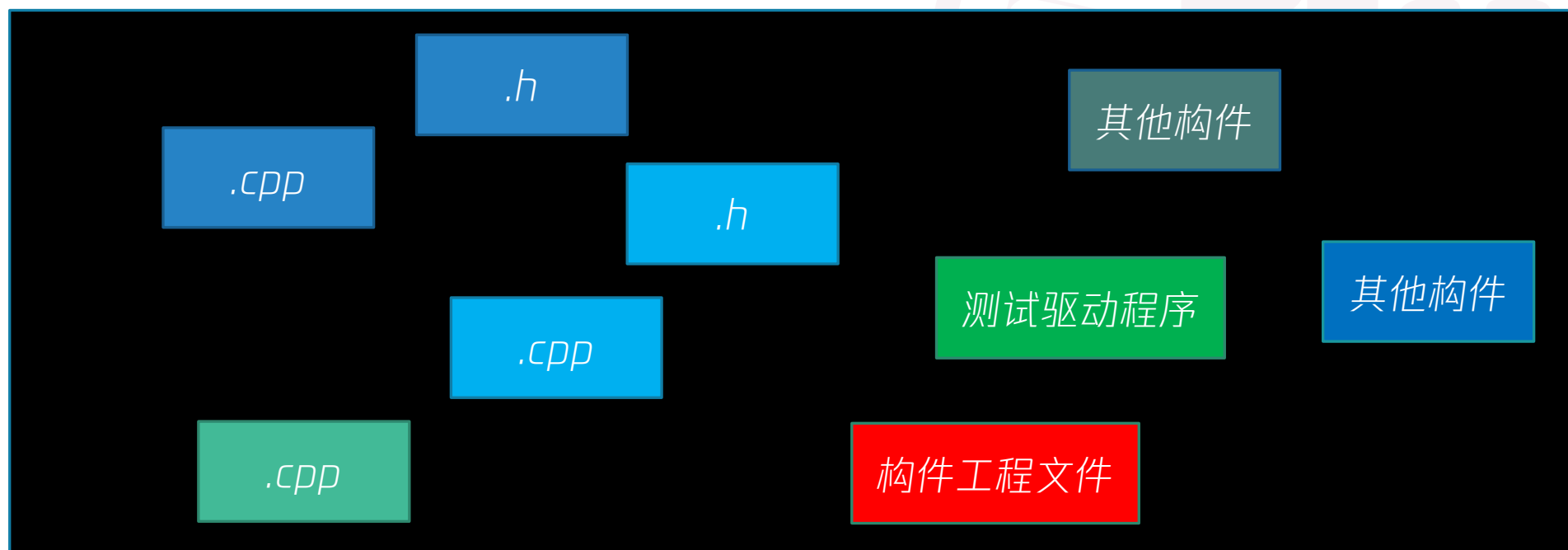
类型

ContentAnalyzer	文件夹
NetworkCardContr	文件夹
OpenSourceProject	文件夹
ResourceLib	文件夹

1. 面向对象编码

步骤2：依次构建各个构件的包含内容

根据面向对象设计阶段完成的构件划分和构件系统架构，依次在每个构件子目录中创建构件包含类的类界面文件和类实现文件。

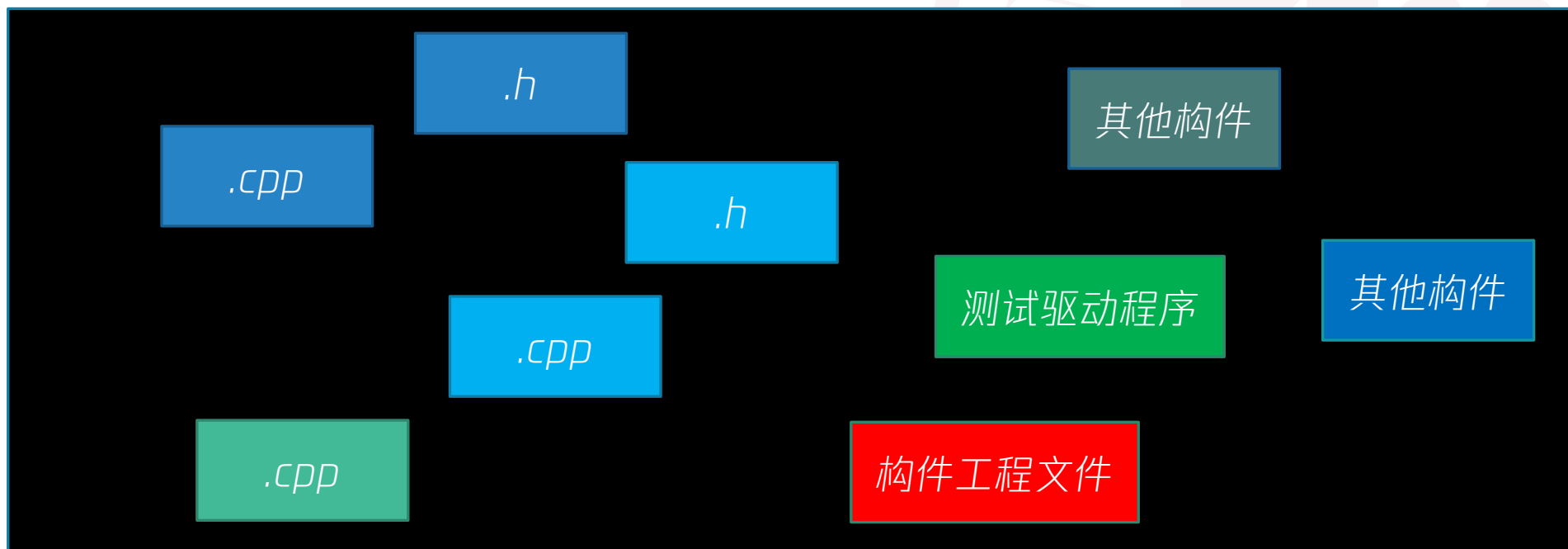


构件子目录

1. 面向对象编码

步骤3：编译并测试软件构件

在集成软件构件时，软件开发人员可以在构件子目录中编写与构件内容相关的测试程序，通过调用构件的功能或内容来测试构件内容是否正确。

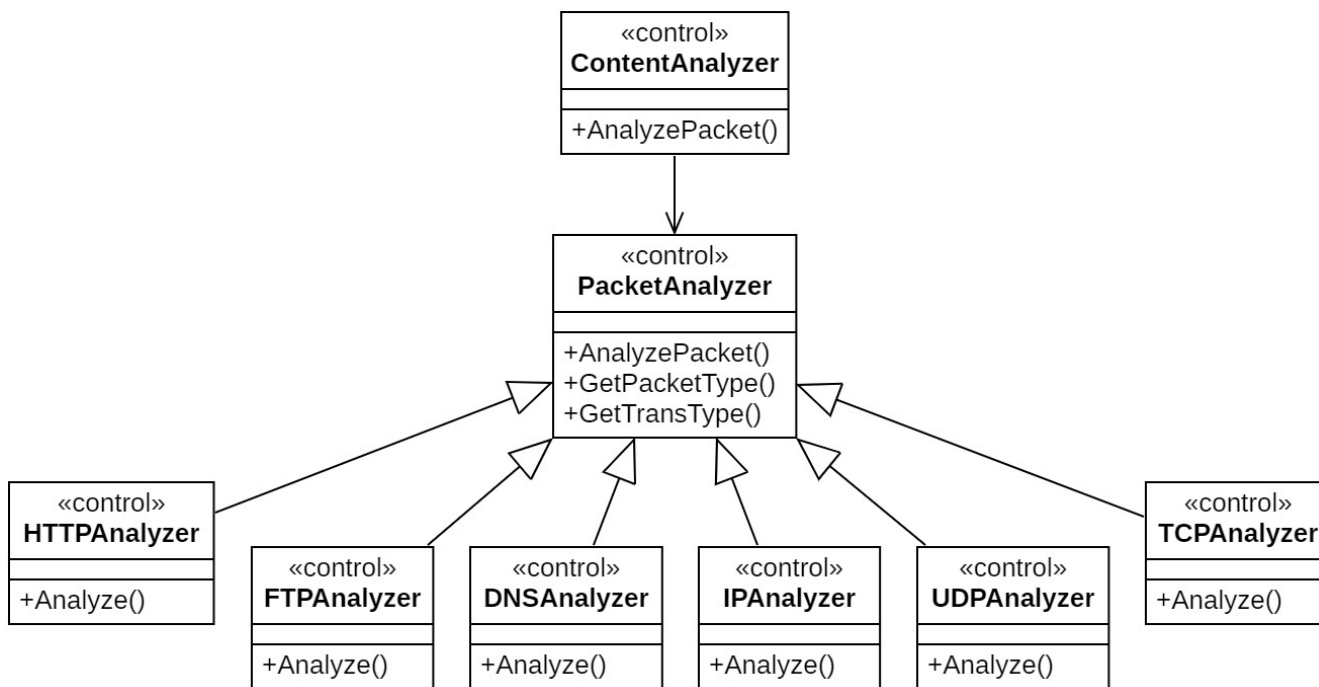


构件子目录

1. 面向对象编码

抓取并分析网络数据包案例

软件构件目录结构实施案例



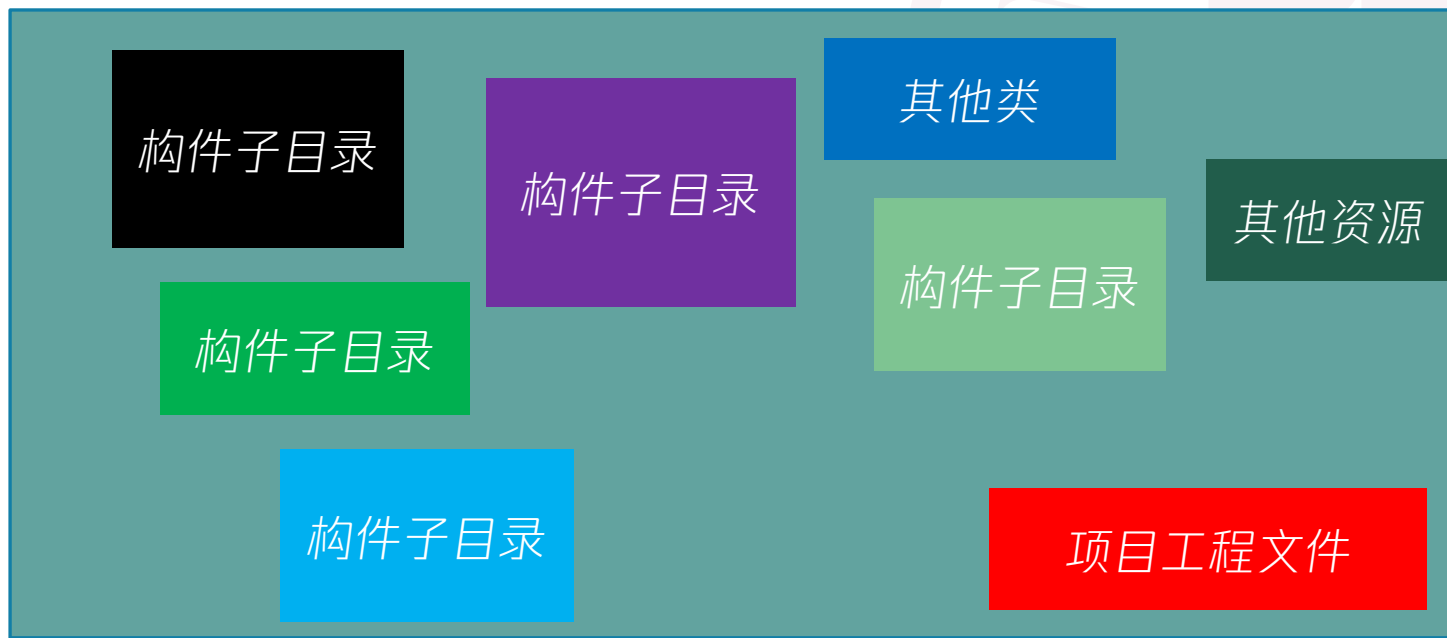
Data (D:) > PacketCaptuer > ContentAnalyzer

名称	类型
ContentAnalyzer	Dev-C++ Project File
ContentAnalyzer.layout	LAYOUT 文件
ContentAnalyzerMain	C++ Source File
ContentAnalyzerTest	C++ Source File
DNSAnalyzer	C++ Source File
DNSAnalyzer	C Header File
FTPAnalyzer	C++ Source File
FTPAnalyzer	C Header File
HTTPAnalyzer	C++ Source File
HTTPAnalyzer	C Header File
IPAnalyzer	C++ Source File
IPAnalyzer	C Header File
PacketAnalyzer	C++ Source File
PacketAnalyzer	C Header File
TCPAnalyzer	C++ Source File
TCPAnalyzer	C Header File
UDPAnalyzer	C++ Source File
UDPAnalyzer	C Header File

1. 面向对象编码

步骤4：构件集成

当目标软件的所有软件构件编译或准备完成以后，软件开发人员可以在软件项目目录中创建目标软件的项目工程文件，通过集成各个软件构件来完成目标软件功能。



项目文件目录

1. 面向对象编码

软件开发人员可以通过编写相应的测试代码，确保目标软件的各个组成部分对项目文件是可用的，且能够被正确集成和调用。



1. 面向对象编码

抓取并分析网络数据包案例

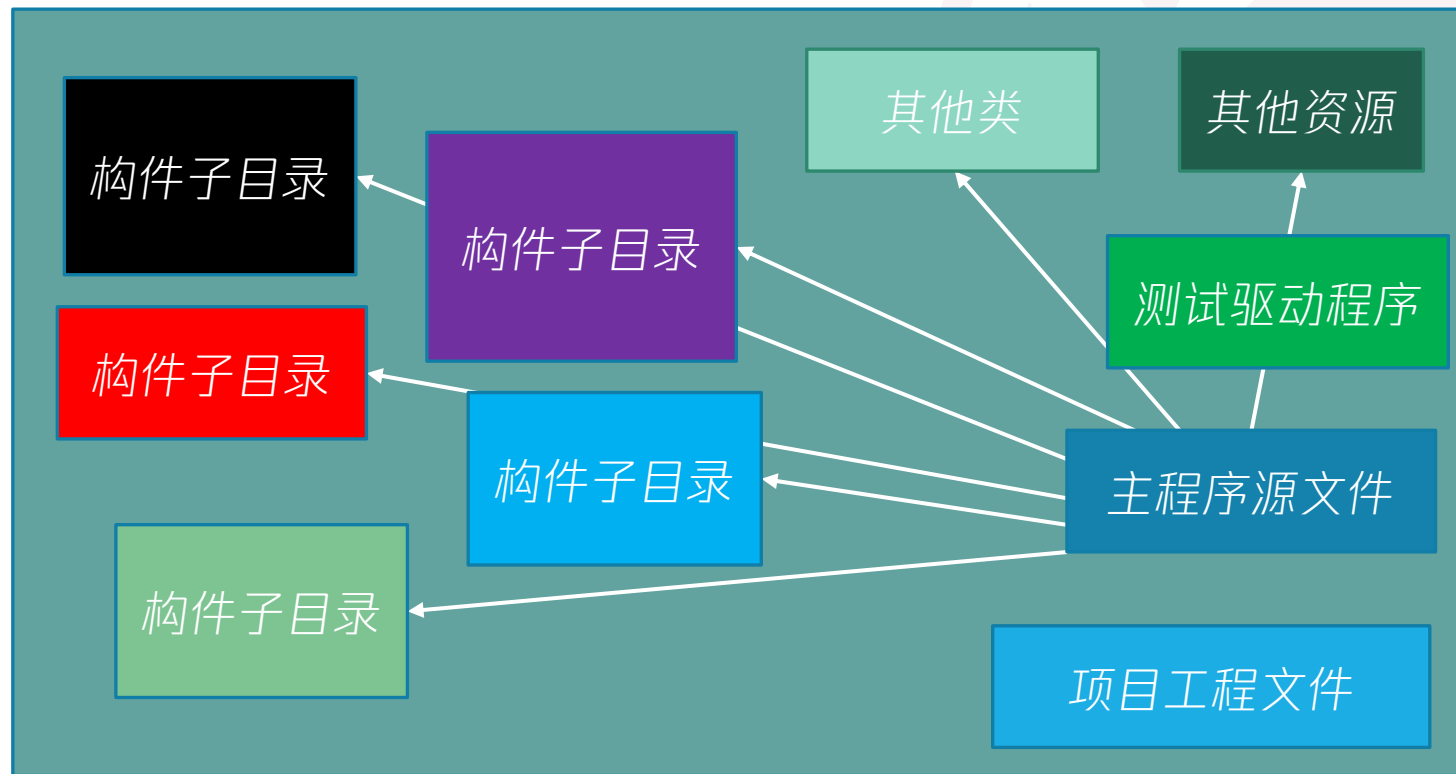
软件构件目录结构实施案例

Data (D:) > PacketCaptuer >		
名称	类型	
ContentAnalyzer	文件夹	
NetworkCardContr	文件夹	
OpenSourceProject	文件夹	
ResourceLib	文件夹	
PacketCaptuer	Dev-C++ Project File	
PacketCaptuer.layout	LAYOUT 文件	
PacketCaptuerMain	C++ Source File	
TestMain	C++ Source File	

1. 面向对象编码

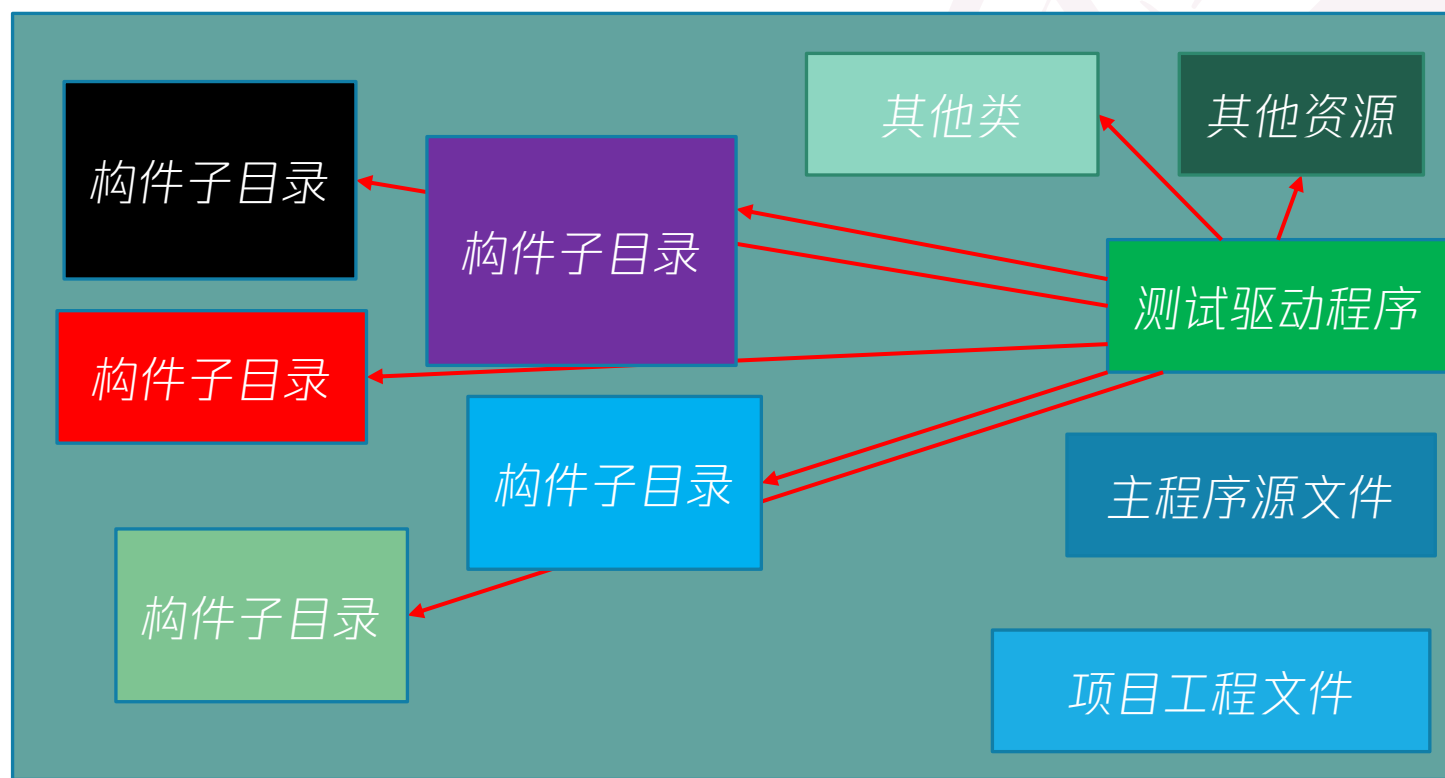
步骤5：目标项目业务实现

按照面向对象设计中的内容来实现目标软件涉及的各项业务功能，在项目主源程序文件中实现目标软件涉及的业务逻辑。



1. 面向对象编码

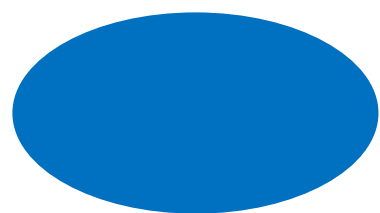
可以在测试源程序文件中编写业务用例测试驱动程序，排除软件业务实现过程中出现的软件bug。



1. 面向对象编码

[2] 面向对象业务实现

根据设计阶段的工作成果，实现用例涉及的业务内容，同时将相关软件模块进行集成。



面向对象分析



软件架构设计



对象交互过程

面向对象设计



业务场景实现



业务场景实现



业务场景实现



业务场景实现



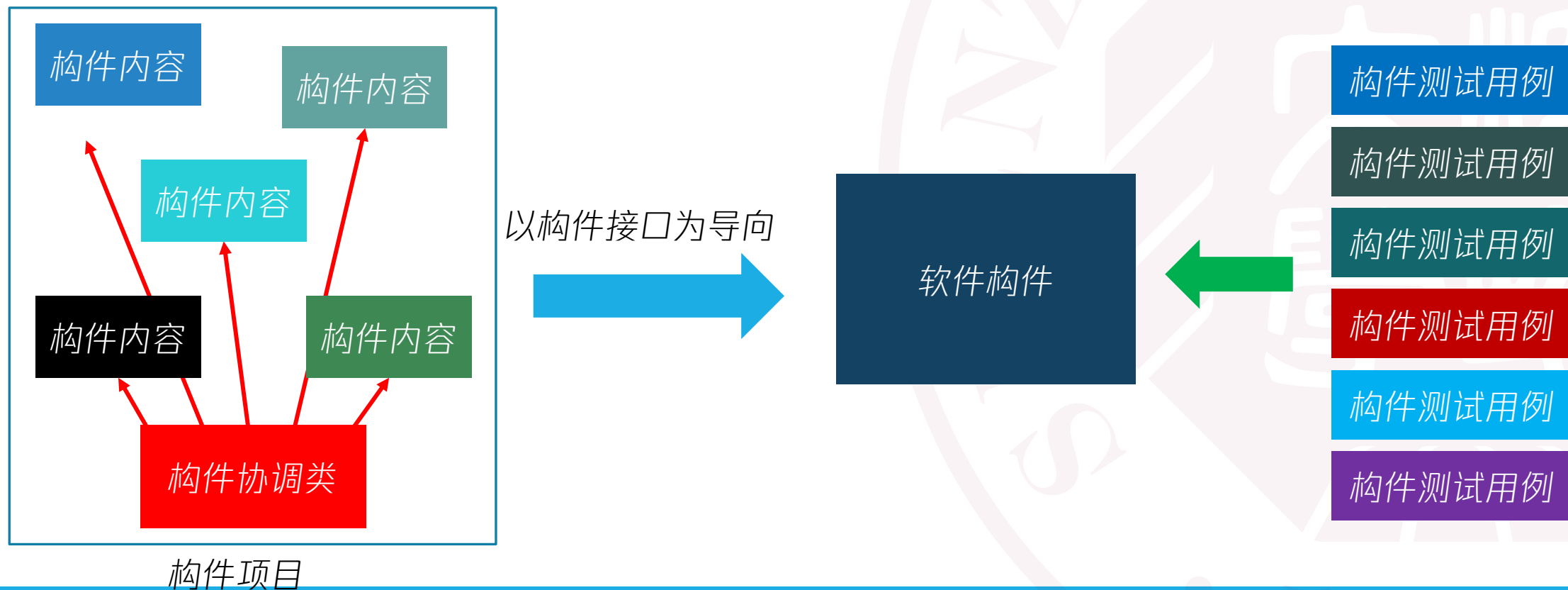
业务场景实现

面向对象实现

1. 面向对象编码

① 构件实现

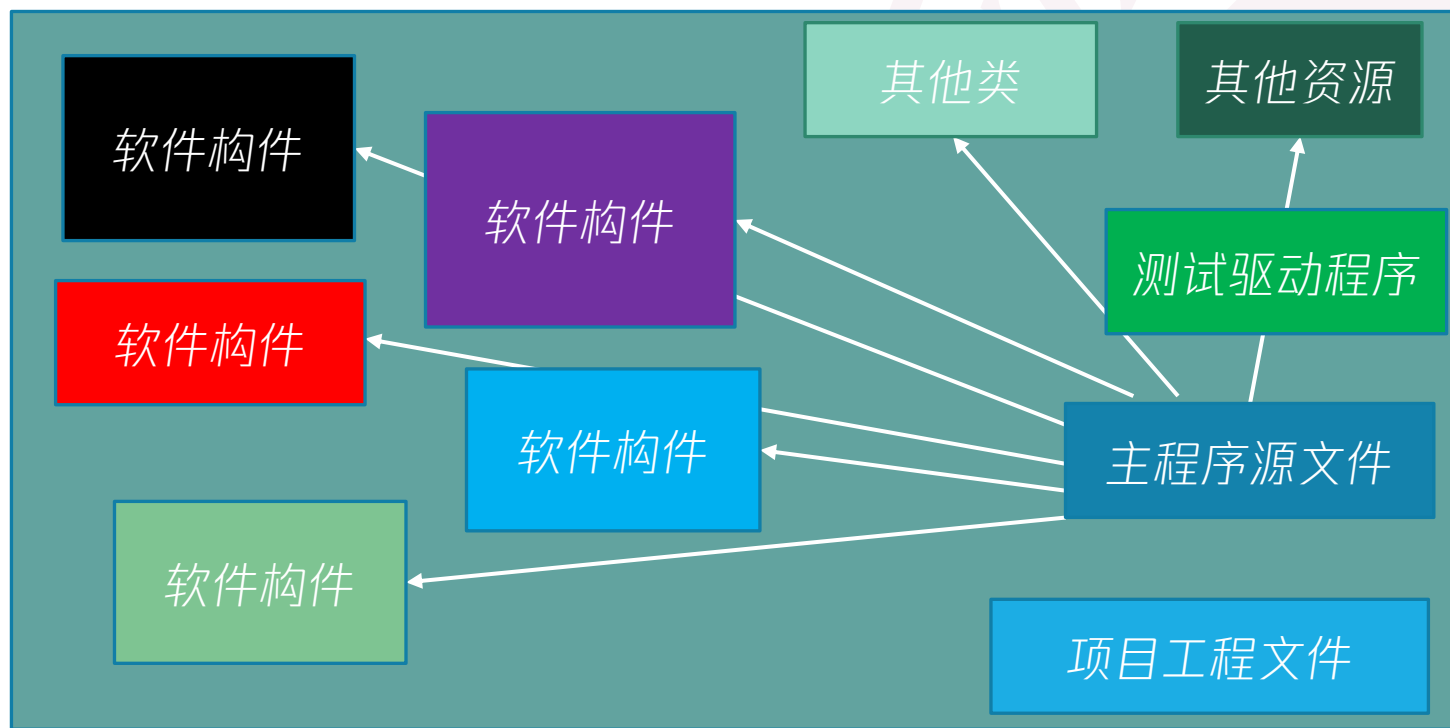
按照面向对象设计阶段约定的业务规则，“以构件接口为导向”来封装构件的各个组成部分。



1. 面向对象编码

② 业务实现

软件开发人员利用得到的软件架构设计，实现用户要求的业务处理流程。



项目文件目录

1. 面向对象编码

[3] 面向对象软件集成

① 传统集成

采用非渐增式组装，或者自底向上集成测试、自顶向下集成测试等渐增方式集成软件模块。

② 协作集成

逐一针对目标软件的具体功能，将相互协作完成该功能的类进行集成。

③ 基于事件〔消息〕的集成

从消息角度出发，渐增地集成与该消息路径相关联的各个构件或类方法，直到所有与消息相关的构件和类被集成为止。

1. 面向对象编码

④ 基于使用的集成

分析类之间的依赖关系，从对其他类依赖最少的类开始，依次将其依赖的类集成到系统中，最后完成整个软件系统的集成。

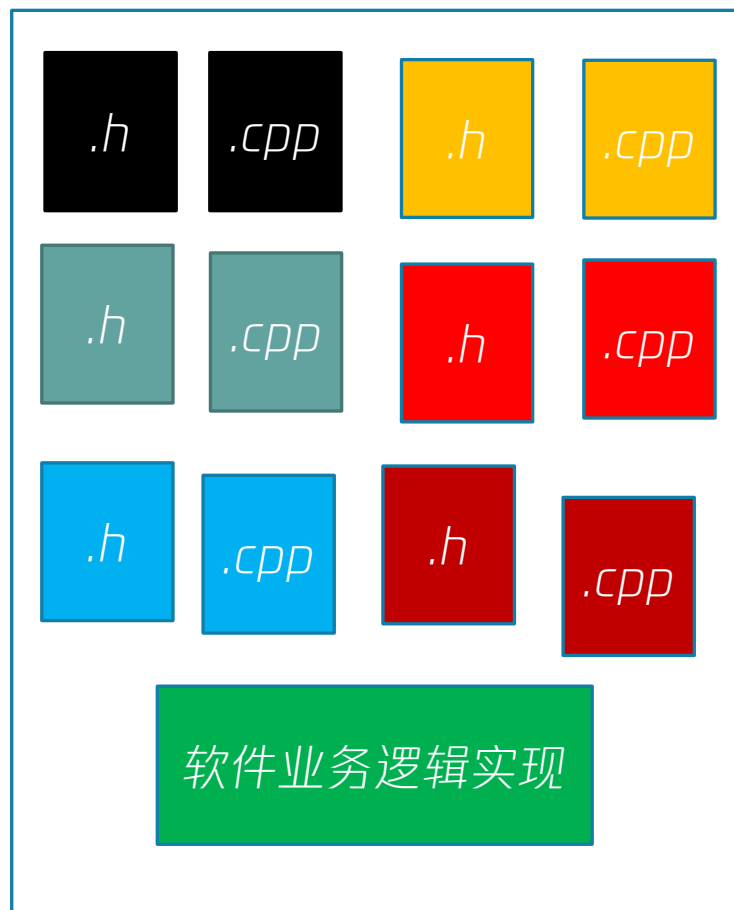
⑤ 客户端/服务器模式集成

从客户端业务开始集成服务器内容。在业务集成初期，可以将服务器需要完成的内容写成桩模块，通过客户端调用，将桩模块逐替换为实际软件模块。

⑥ 分布式集成

分别集成各个软件构件。构件之间通过消息或者信令来提供目标服务。

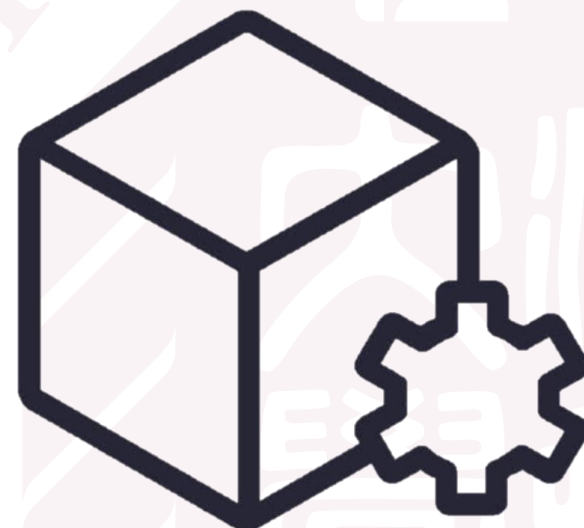
1. 面向对象编码



面向对象软件项目

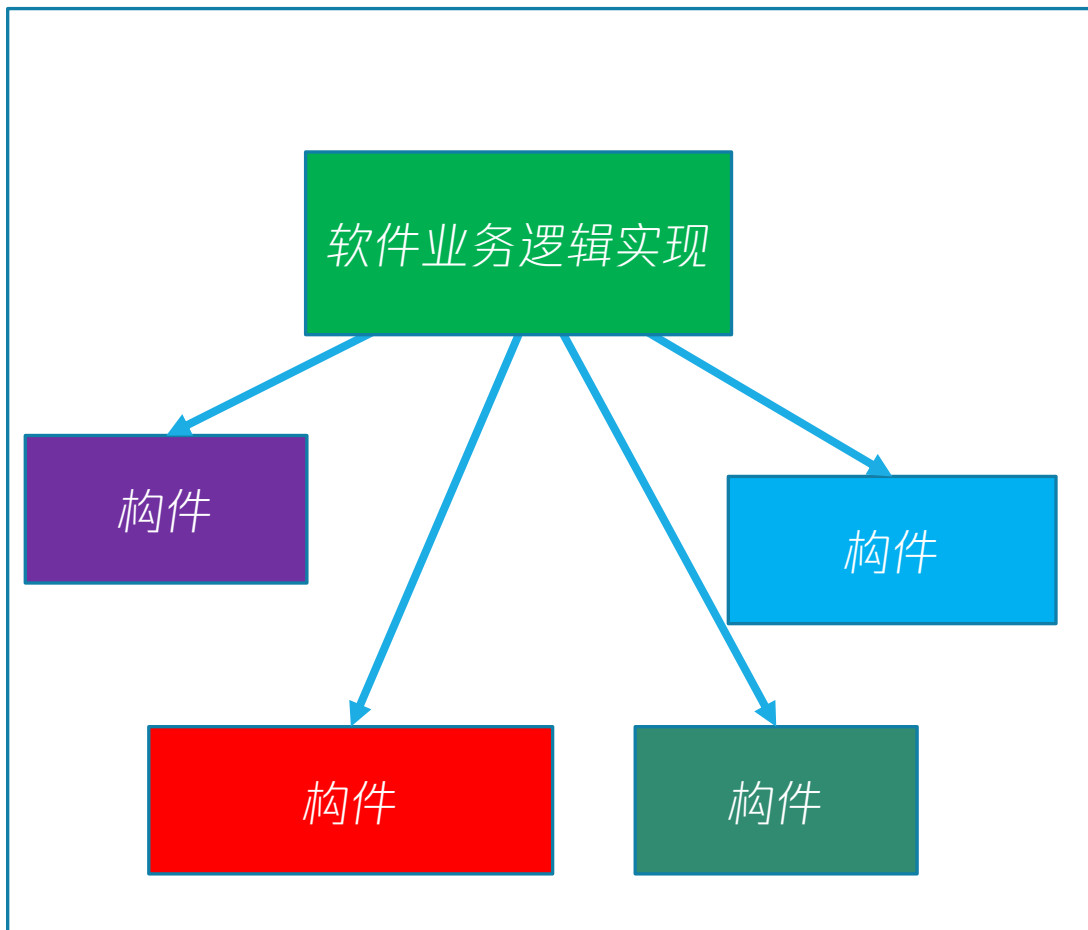
对象交互
脚本设计

软件架构设计



目标软件

1. 面向对象编码

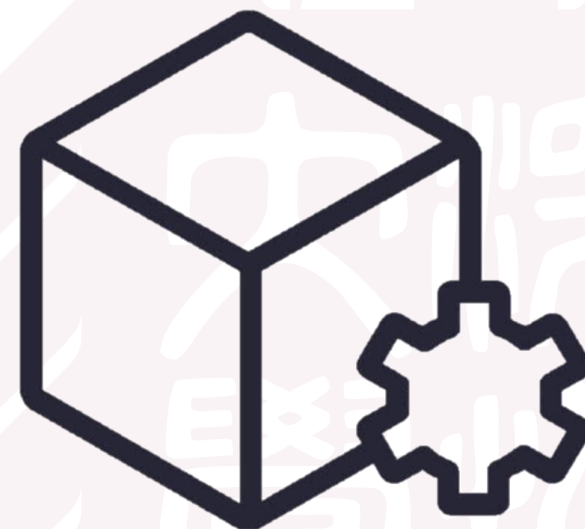


面向对象软件项目

对象交互
脚本设计



软件架构设计



目标软件

2. 面向对象测试

面向对象方法中可能导致的缺陷：

- 私有方法无法访问
- 缺陷被继承
- 多态导致问题复杂
- 缺陷由多个类共同促成

2. 面向对象测试

[1] 面向对象测试阶段

针对面向对象分析的测试

针对面向对象设计的测试

针对面向对象编程的测试

针对面向对象软件

2. 面向对象测试

① 针对面向对象分析的测试

对确定的类候选者进行测试

对确定的类关联进行测试

对主题进行测试

对定义类属性和实例关联进行测试

对定义类服务和消息关联进行测试

2. 面向对象测试

② 针对面向对象设计的测试

对类设计的测试

对软件架构设计的测试

对业务流的测试

对软件构件的测试

对软件部署模式的测试

2. 面向对象测试

③ 针对面向对象编程的测试

检测完成的代码是否与设计结果相符

检测构件封装是否正确

检测完成的代码风格是否符合要求

2. 面向对象测试

④ 针对面向对象软件的测试

功能是否符合用户需求

性能是否符合用户需求

软件配置是否完整

2. 面向对象测试

(2) 面向对象测试策略



2. 面向对象测试

① 面向对象单元测试

- 借助结构化测试中的白盒测试和黑盒测试为类方法设计测试用例
- 针对类的属性和方法设计其他单元测试用例

② 面向对象集成测试

将多个类中相互依赖的方法联合参与测试，确保类之间的交互能够满足设计要求。

2. 面向对象测试

根据测试的目标不同，面向对象集成测试可以分为静态测试和动态测试两个步骤。

1. 静态测试

- 确保完成的软件架构符合设计要求

2. 动态测试

- 结合使用场景，参考软件架构来设计测试用例

2. 面向对象测试

③ 面向对象确认测试

- 有效性测试
- 验收测试
- 检测配置是否完整

④ 面向对象系统测试

检测开发出来的软件能否达到用户需求。

2. 面向对象测试

[3] 面向对象测试用例设计

① 单元测试

测试的目标是组成目标软件的各个类。

软件测试人员除了可以采用结构化方法中的白盒测试或黑盒测试为类中的方法设计测试用例以外，还需要结合面向对象程序设计的特点，对类的属性和方法开展综合测试。

软件测试人员需要寻找目标待测类的不变式，在不变式的基础上结合特定的标准为目标待测类设计测试用例。

2. 面向对象测试

面向对象单元测试案例

*SampleClass*类

不变式: Initialize().Finalize()

操作系列:

Initialize().

[GetAttribute()|SetAttribute()|UpdateAttribute()|MethodWithAttribute()|MethodWithoutAttribute()]ⁿ.

Finalize()

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

2. 面向对象测试

面向对象单元测试用例设计方法



2. 面向对象测试

1) 随机测试

在目标待测类的最小操作集中随机引入 $0 \sim n$ 个其他操作，产生一系列随机测试用例。

*SampleClass*类的操作系列:

Initialize().[GetAttribute()|SetAttribute()|UpdateAttribute()|MethodWithAttribute()|MethodWithoutAttribute()]ⁿ.Finalize()

测试用例r1:

Initialize().GetAttribute().SetAttribute().GetAttribute().GetAttribute().Finalize()

测试用例r2:

Initialize().SetAttribute().UpdateAttribute().MethodWithAttribute().Finalize()

2. 面向对象测试

2) 划分测试



2. 面向对象测试

➤ 基于状态的划分

根据类操作是否改变类的状态〔属性〕来划分类操作，并在最小操作集的基础上设计测试用例。

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

会改变类状态的操作

SetAttribute()

UpdateAttribute()

不会改变类状态的操作

GetAttribute()

MethodWithAttribute()

MethodWithoutAttribute()

2. 面向对象测试

测试用例p1 [改变类状态的操作1] :

Initialize().SetAttribute().Finalize()

测试用例p2 [改变类状态的操作2] :

Initialize().UpdateAttribute().Finalize()

测试用例p3 [改变类状态的操作3] :

Initialize().UpdateAttribute().SetAttribute().Finalize()

测试用例p4 [不改变类状态的操作] :

Initialize().GetAttribute().Finalize()

测试用例p5 [不改变类状态的操作] :

Initialize().MethodWithAttribute().Finalize()

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

2. 面向对象测试

➤ 基于属性的划分

根据类操作对属性的操作方式来划分类操作，并在最小操作集的基础上设计测试用例。

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

使用属性的操作:

GetAttribute()

MethodWithAttribute()

修改属性的操作:

SetAttribute()

UpdateAttribute()

不使用也不修改属性的操作:

MethodWithoutAttribute()

2. 面向对象测试

测试用例a1 [使用属性的操作1] :

Initialize().GetAttribute().Finalize()

测试用例a2 [使用属性的操作2] :

Initialize().MethodWithAttribute().Finalize()

测试用例a3 [使用属性的操作3] :

Initialize().GetAttribute().MethodWithAttribute().Finalize()

测试用例a4 [修改属性的操作1] :

Initialize().SetAttribute().Finalize()

测试用例a5 [修改属性的操作2] :

Initialize().UpdateAttribute().Finalize()

测试用例a6 [修改属性的操作3] :

Initialize().SetAttribute().UpdateAttribute().Finalize()

测试用例a7 [不使用也不修改属性的操作] :

Initialize().MethodWithoutAttribute ().Finalize()

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

2. 面向对象测试

➤ 基于功能的划分

根据类操作完成的功能对类方法进行分类，并为每个分类设计对应的测试用例。

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

查询操作:

GetAttribute()

MethodWithAttribute()

修改操作:

SetAttribute()

UpdateAttribute()

其他操作:

MethodWithoutAttribute()

2. 面向对象测试

测试用例m1 [查询操作1] :

Initialize().GetAttribute().Finalize()

测试用例m2 [查询操作2] :

Initialize().MethodWithAttribute().Finalize()

测试用例m3 [修改操作] :

Initialize().SetAttribute().UpdateAttribute().Finalize()

测试用例m4 [其他操作] :

Initialize().MethodWithoutAttribute().Finalize()

SampleClass
+Attribute +WorkState
+Initialize() +GetAttribute() +SetAttribute() +UpdateAttribute() +MethodWithAttribute() +MethodWithoutAttribute() +Finalize()

2. 面向对象测试

3) 基于故障的测试

结合常见的错误和软件测试经验来推测软件中可能存在的错误，并根据推测内容来设计测试用例。

2. 面向对象测试

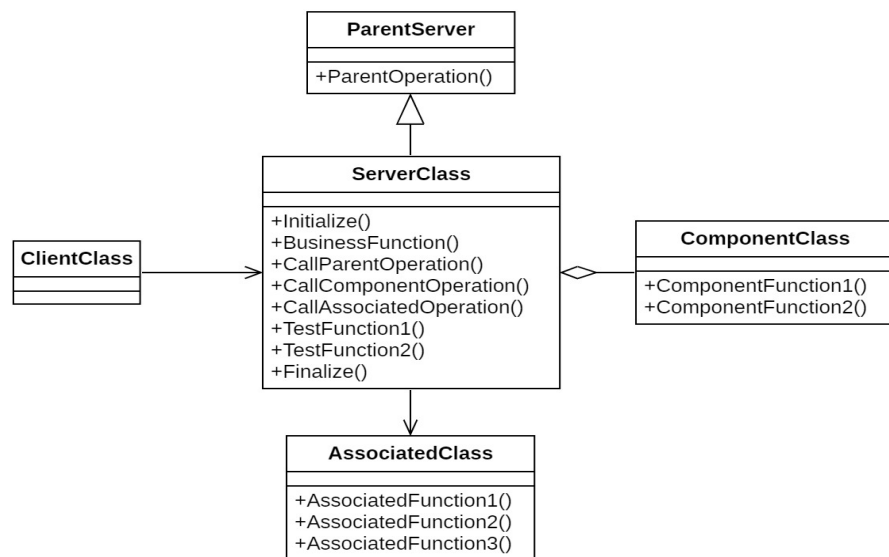
② 集成测试

在面向对象集成过程中，软件测试人员必须按照特定的模块集成规则将多个类进行集成。此时，参与集成测试的类的数量可能超过一个，前期的单元测试方式无法继续满足集成测试的要求。

软件测试人员除了可以继续使用随机测试、划分测试为集成后的多个类设计测试用例以外，还可以结合业务场景和业务逻辑对集成后的多个类开展测试。

2. 面向对象测试

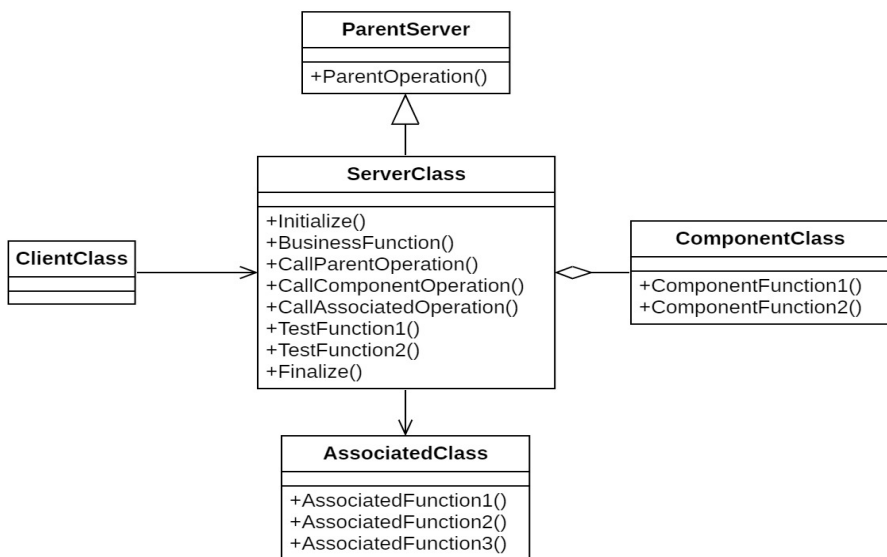
面向对象集成测试案例



*ServerClass*类最小操作集合:
Initialize().Finalize()

2. 面向对象测试

面向对象集成测试案例



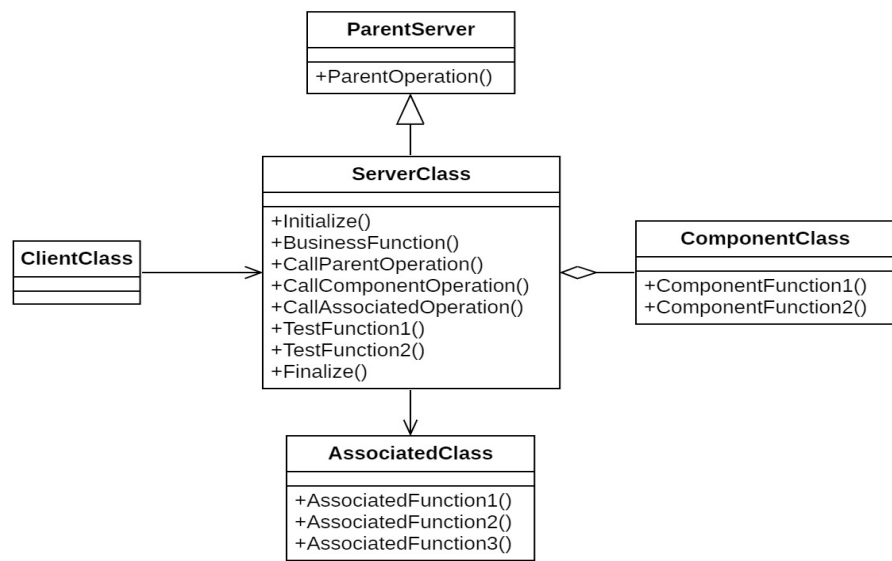
最小操作集合:

`Initialize().[BusinessFunction()|CallParentOperation()|
CallComponentOperation()|CallAssociatedOperation()|
TestFunction1()|TestFunction2()]n.Finalize()`

2. 面向对象测试

1) 集成随机测试

根据随机产生的操作系列来确定参与测试的类和类操作。



最小操作集合:

`Initialize().[BusinessFunction()|CallParentOperation()|
CallComponentOperation()|CallAssociatedOperation()|
TestFunction1()|TestFunction2()]n.Finalize()`

测试用例r1:

`Initialize().TestFunction1().Finalize()`

测试用例r2:

`Initialize().CallParentOperation().Finalize()`

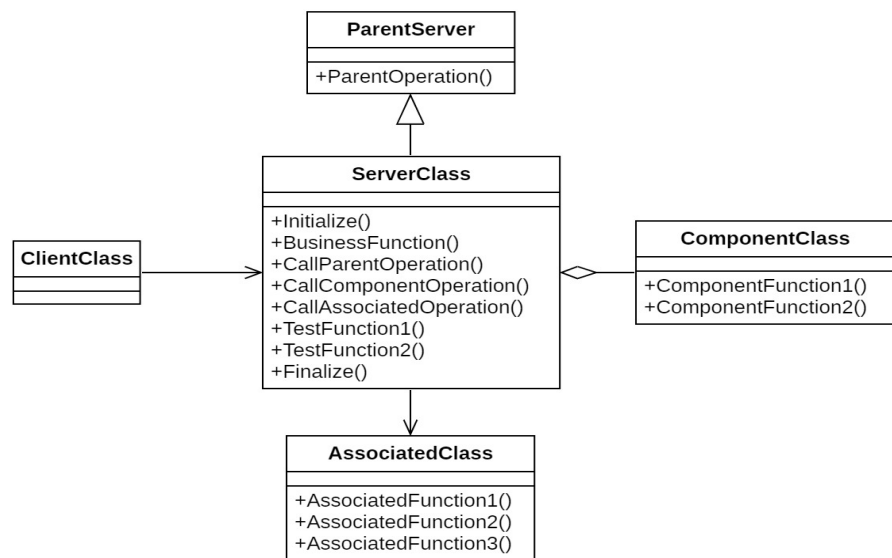
测试用例r3:

`Initialize().CallAssociatedOperation().Finalize()`

2. 面向对象测试

1) 集成随机测试

根据随机产生的操作系列来确定参与测试的类和类操作。



最小操作集合:

`Initialize().[BusinessFunction()|CallParentOperation()|
CallComponentOperation()|CallAssociatedOperation()|
TestFunction1()|TestFunction2()]n.Finalize()`

测试用例r2:

`Initialize().CallParentOperation().Finalize()`



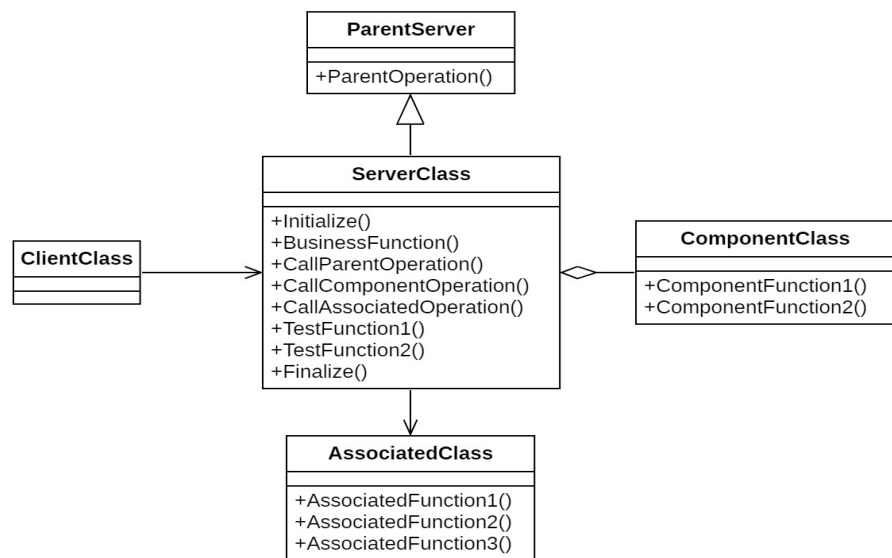
测试用例r2:

`Initialize()ServerClass.CallParentOperation()ServerClass.ParentOperation()ParentServer.Finalize()ServerClass`

2. 面向对象测试

1) 集成随机测试

根据随机产生的操作系列来确定参与测试的类和类操作。



最小操作集合:

`Initialize().[BusinessFunction()|CallParentOperation()|
CallComponentOperation()|CallAssociatedOperation()|
TestFunction1()|TestFunction2()]n.Finalize()`

测试用例r3:

`Initialize().CallAssociatedOperation().Finalize()`



测试用例r3:

`Initialize()ServerClass.CallAssociatedOperation()ServerClass. [AssociatedFunction1()AssociatedClass.
AssociatedFunction2()AssociatedClass].Finalize()ServerClass`

2. 面向对象测试

2) 集成划分测试

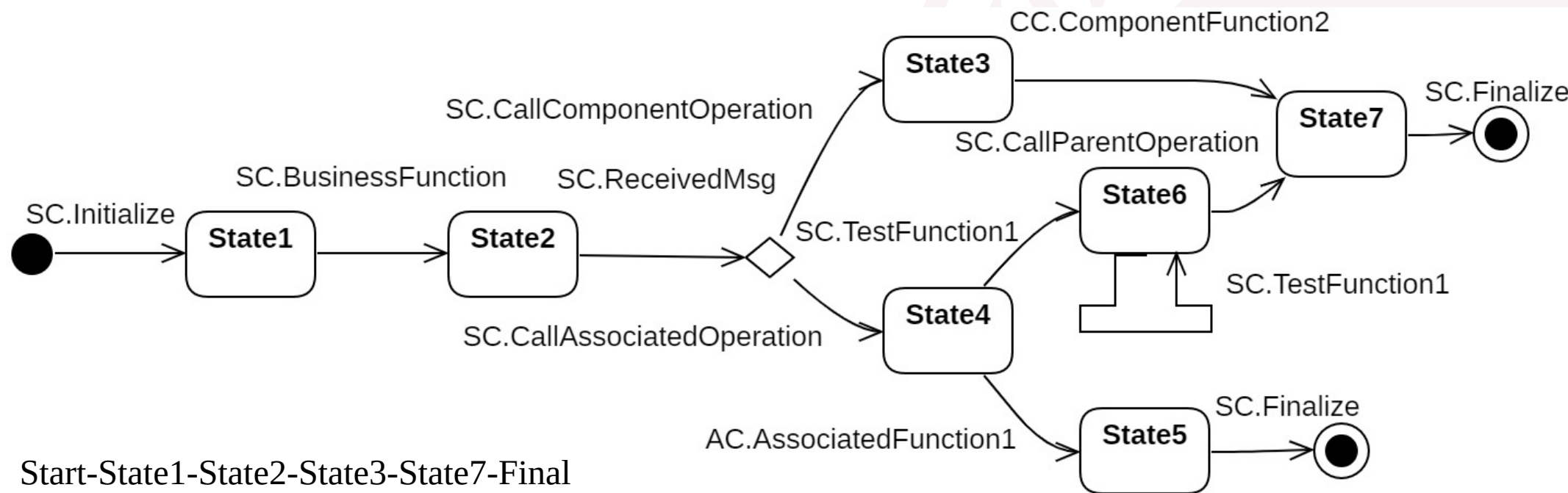
结合目标待测案例的状态、属性和功能等多个因素来设计测试用例。



2. 面向对象测试

3) 由状态图导出测试用例

借助系统状态图，从系统状态变迁序列中设计测试用例。



Path1: Start-State1-State2-State3-State7-Final

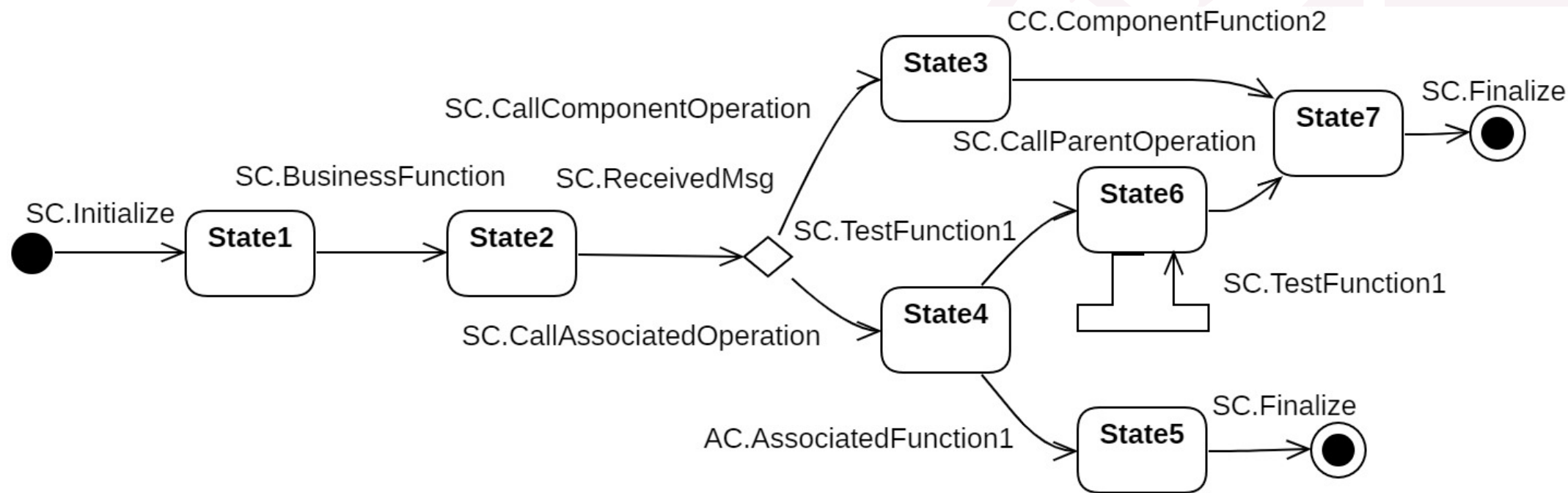
Path2: Start-State1-State2-State4-State6-State7-Final

Path3: Start-State1-State2-State4-State6-State6-State7-Final

Path4: Start-State1-State2-State4-State5-Final

2. 面向对象测试

3) 由状态图导出测试用例



Path1: Start-State1-State2-State3-State7-Final

测试用例SP1 [覆盖路径Path1] :

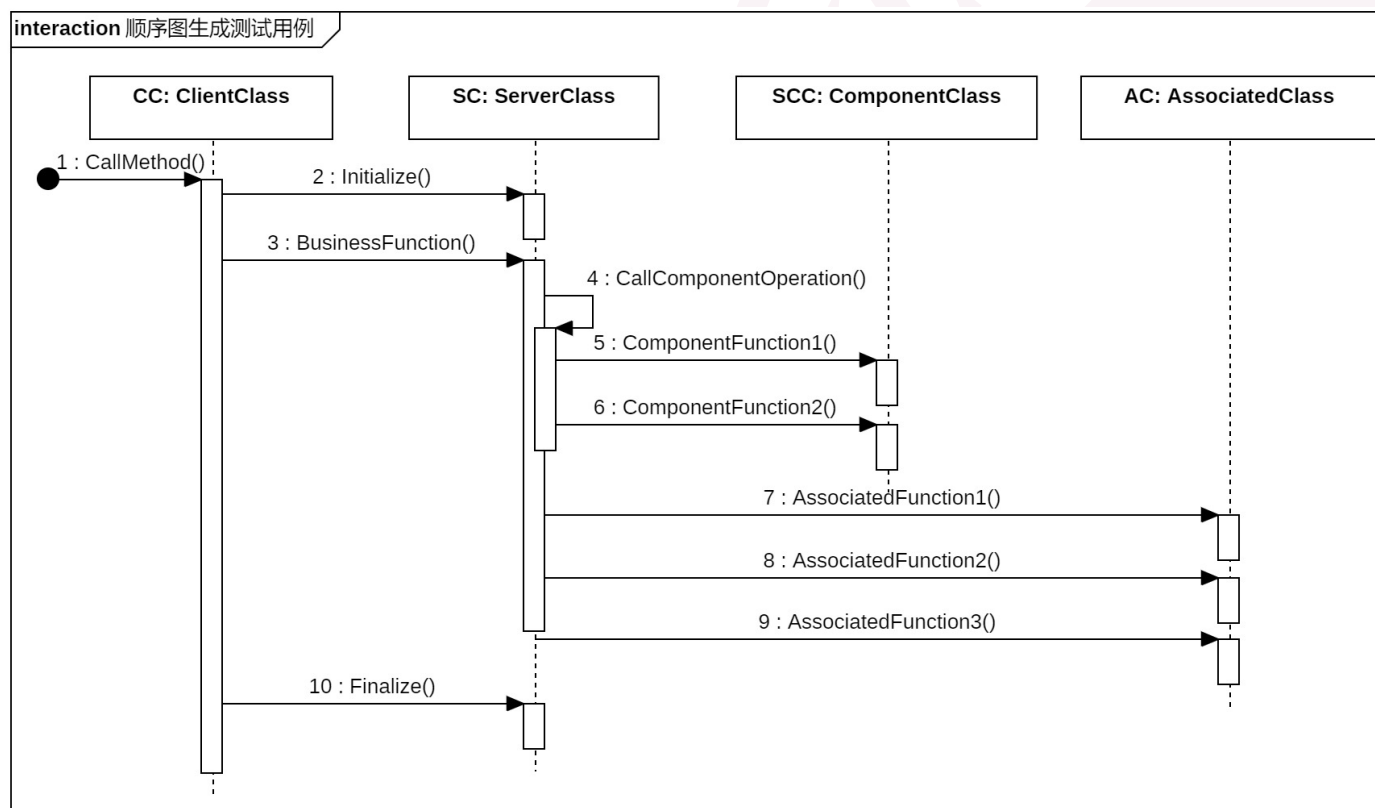
Initialize_{ServerClass}.BusinessFunction_{ServerClass}.ReceivedMsg_{ServerClass}.CallComponentOperation_{ServerClass}.

ComponentFunction2_{ComponentClass}.Finalize_{ServerClass}

2. 面向对象测试

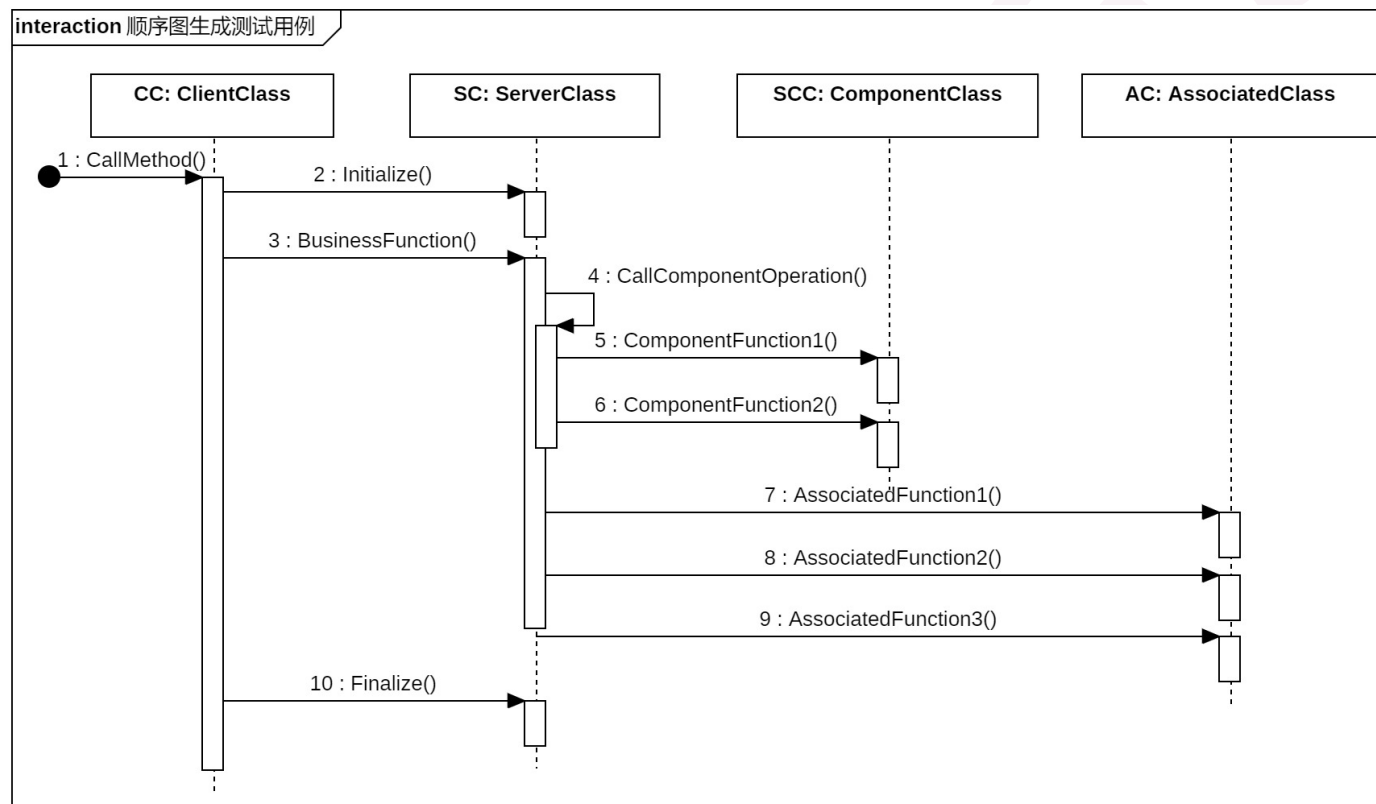
4) 由顺序图导出测试用例

借助顺序图，跟踪顺序图中的消息序列来生成满足各个用例实现过程的测试用例。



2. 面向对象测试

4) 由顺序图导出测试用例



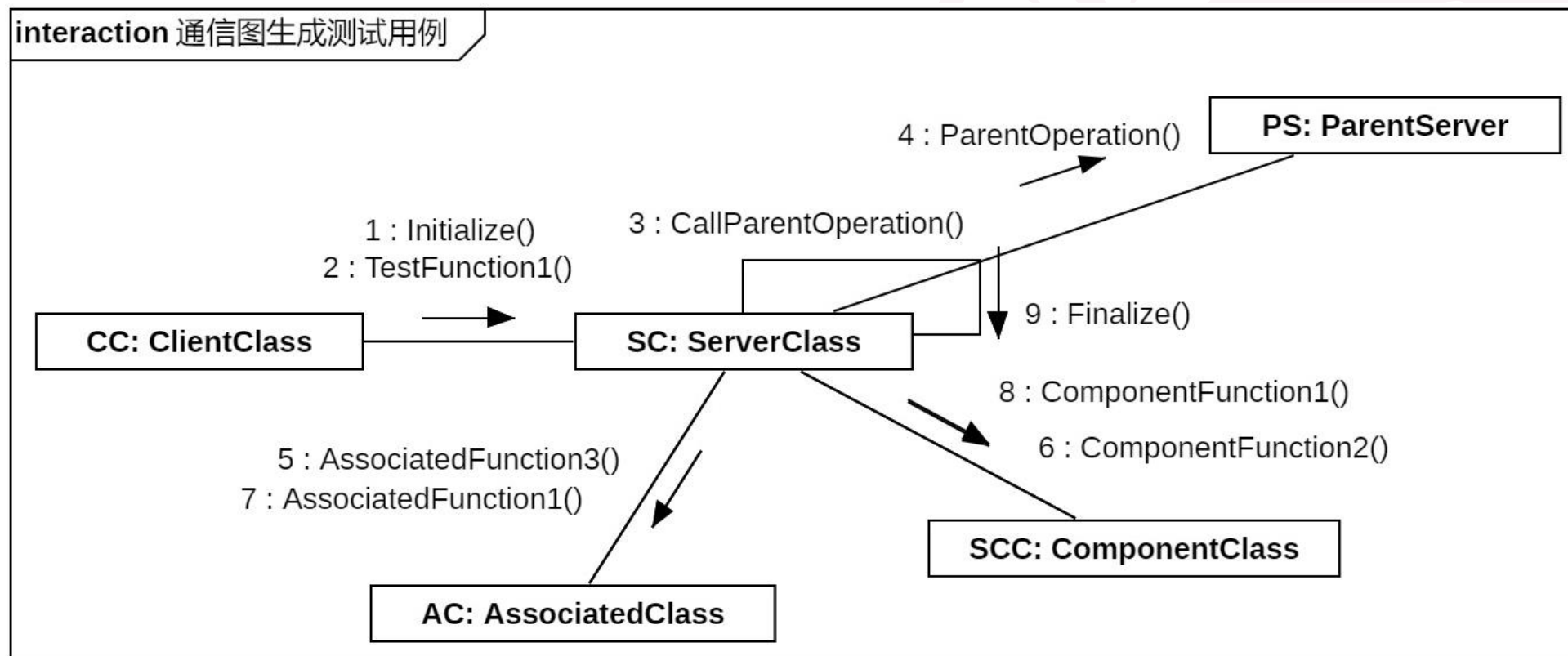
测试用例5Q1:

Initialize_{ServerClass}.BusinessFunction_{ServerClass}.CallComponentOperation_{ServerClass}.ComponentFunction1_{ComponentClass}.
ComponentFunction2_{ComponentClass}.AssociatedFunction1_{AssociatedClass}.AssociatedFunction2_{AssociatedClass}.
AssociatedFunction3_{AssociatedClass}.Finalize_{ServerClass}

2. 面向对象测试

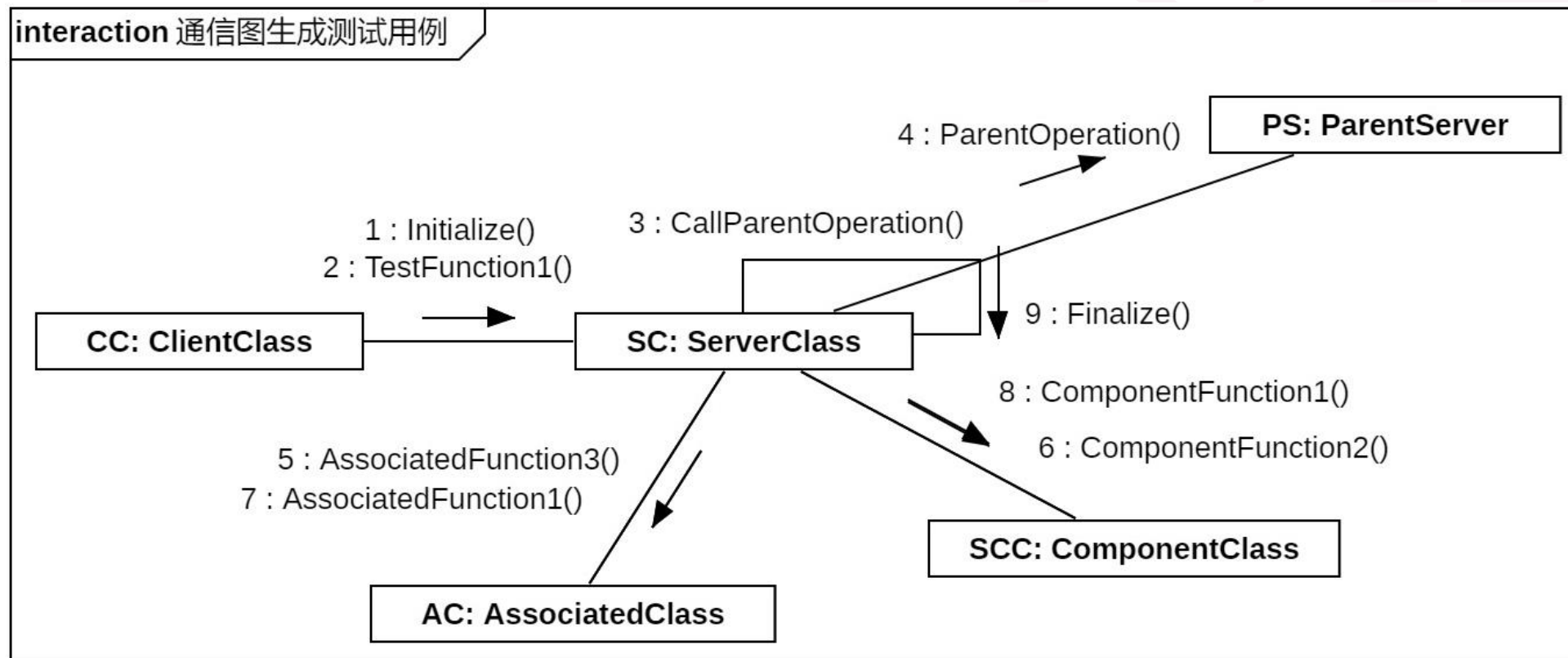
5) 由通信图导出测试用例

借助通信图，通过跟踪对象之间的交互过程来生成测试复杂业务逻辑的测试用例，从代码层面对复杂业务逻辑进行详细测试。



2. 面向对象测试

5) 由通信图导出测试用例



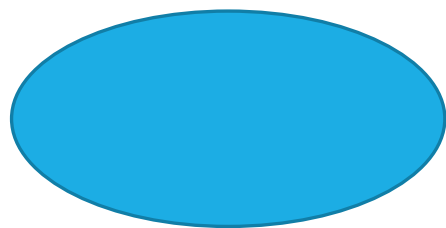
测试用例C01:

Initialize_{ServerClass}.TestFunction1_{ServerClass}.CallParentOperation_{ServerClass}.ParentOperation_{ParentServer}.AssociatedFunction3_{AssociatedClass}.
ComponentFunction2_{ComponentClass}.AssociatedFunction1_{AssociatedClass}.ComponentFunction1_{ComponentClass}.Finalize_{ServerClass}

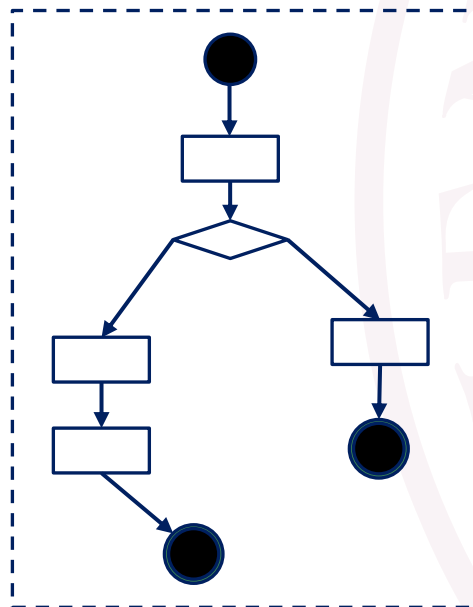
2. 面向对象测试

6) 基于场景的测试

结合面向对象分析内容，从业务用例涉及的活动生成测试用例。



用例



活动图



测试用例

测试用例

测试用例

测试用例

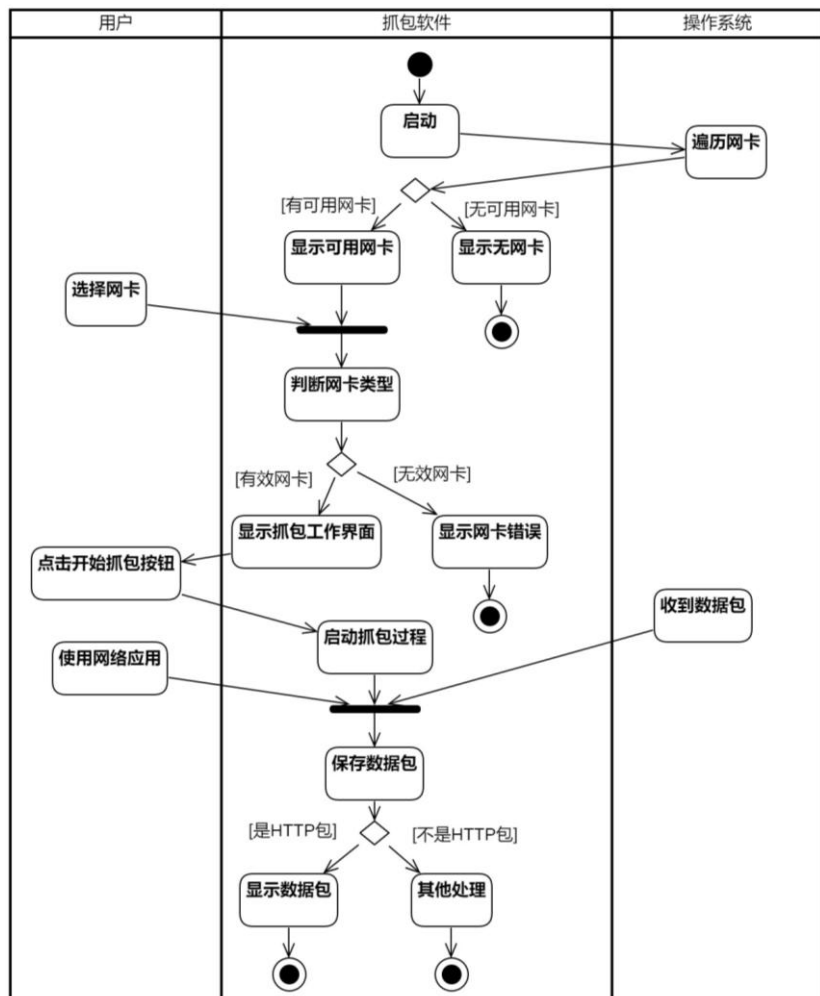
测试用例

测试用例

2. 面向对象测试

抓取并分析网络数据包案例

面向对象集成场景测试案例



测试用例A1〔无网卡退出〕：

在计算机中禁止所有网卡〔或者台式机拔出网卡〕
运行抓包分析软件
系统遍历网卡，并显示无网卡退出

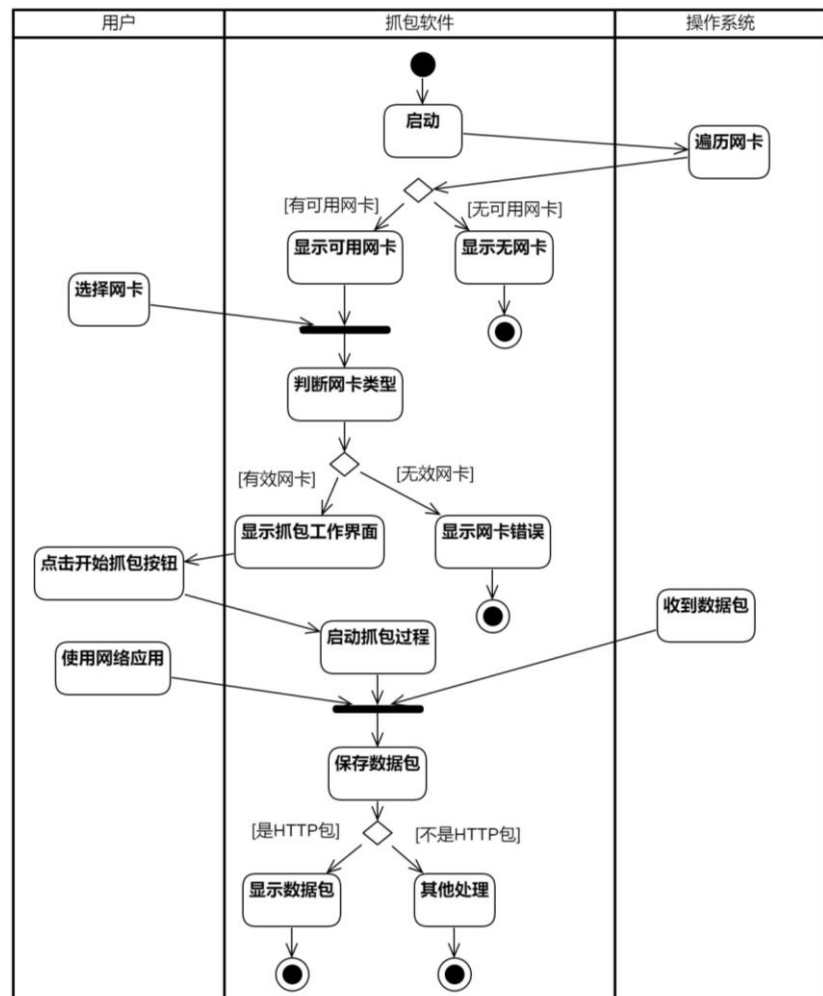
测试用例A2〔显示网卡错误〕：

用户启用网卡，且确保电脑中有多块网卡
用户运行抓包分析软件
系统遍历网卡，显示可用网卡
用户选择不可抓包网卡
系统判断网卡无效，提示无网卡退出

2. 面向对象测试

抓取并分析网络数据包案例

面向对象集成场景测试案例



测试用例A3〔显示数据包〕:

用户运行抓包分析软件
系统遍历网卡，显示可用网卡
用户选择可以抓包的网卡
系统显示抓包工作界面
用户点击开始抓包
系统启用抓包过程
用户进行网页浏览
系统显示抓到的HTTP包内容

测试用例A4〔其他处理〕:

用户运行抓包分析软件
系统遍历网卡，显示可用网卡
用户选择可以抓包的网卡
系统显示抓包工作界面
用户点击开始抓包
系统启用抓包过程
用户进行非网页浏览应用，例如：QQ聊天，FTP等
系统继续运行，无提示

2. 面向对象测试

7) 基于异常、故障的测试

通过推测软件中可能存在的错误，针对推测的错误设计测试用例。

在面向对象方法中，由于对异常、故障的处理可以被放到不同的类中，因此，软件测试人员在设计测试用例时必须尽可能地覆盖类间隐式的依赖关系。

3. 小结

在设计转换为实现的过程中，程序设计人员需要结合面向对象特点对目标软件的程序源代码进行组织和集成；

通过面向对象测试，软件测试人员可以：

- [1] 排查出软件实现与面向对象分析与设计的不一致性；
- [2] 确保完成的软件系统能够提供完整、正确的业务功能。

3. 小结

面向对象测试除了需要兼顾结构化测试涉及的内容以外，还需要结合面向对象特点来开展特殊的单元测试和集成测试。

[1] 可以根据面向对象设计阶段形成的软件架构设计、对象图、顺序图、状态图等内容设计目标系统的白盒测试用例；

[2] 可以根据面向对象分析，结合用例包含内容及活动为待测系统设计黑盒测试用例。

谢谢大家!

课后思考:

1. 在实现过程中，如何做到对系统的充分测试？
2. 面向对象的白盒测试和结构化的白盒测试有什么区别？
3. 怎样的设计成果才能更好地帮助软件实现？